🚀 Collections Framework Masterclass

Chapter 2 — Collection Interface (Deep Dive)

> **Interview Importance:** ⭐⭐⭐⭐⭐

This is one of the most fundamental chapters in Java Collections. Every collection class (except `Map`) is built upon the `Collection` interface. Understanding this interface means understanding the foundation of the entire Collections Framework.

What is Collection Interface?

`Collection` is the **root interface** of the Java Collections Framework.

It represents a **group of objects** known as **elements**.

Package:

```
java.util.Collection
```

Declaration:

```
public interface Collection<E> extends Iterable<E>
```

Notice:

- Collection extends `Iterable`
- Therefore every Collection object can be traversed using
- Iterator
- Enhanced for-loop
- Stream API

Position in Hierarchy

```
                Iterable
                    ↑
              Collection
```

```
          _________|_________
         |         |         |
      List       Set       Queue
        |
  ArrayList
  LinkedList
  Vector
```

Map is **NOT** part of this hierarchy.

Why Collection is an Interface?

Interview Question ⭐⭐⭐⭐⭐

Suppose Java had made Collection a class.

Then every implementation would inherit the same behavior.

But different collections have different behaviors.

Example

```
ArrayList

Allows duplicates

Maintains insertion order
```

```
HashSet

No duplicates

No ordering
```

```
PriorityQueue

Priority-based ordering
```

Hence Java defined **rules** (interface), and each implementation provides its own behavior.

Collection Interface Methods

There are approximately **15 core methods** every Java developer should know.

We'll cover each one in detail.

---

1. add()

Signature

```
boolean add(E e)
```

Adds an element.

Returns

```
true
```

if collection changes.

Example

```
Collection<String> names = new ArrayList<>();

names.add("Rahul");
names.add("John");
```

Output

```
[Rahul, John]
```

Interview Question

Why does add() return boolean?

Because some collections may reject insertion.

Example

HashSet

```
Set<Integer> set = new HashSet<>();

set.add(10);
set.add(10);
```

Second insertion

returns

```
false
```

because duplicate wasn't added.

---

2. addAll()

Signature

```
boolean addAll(Collection<? extends E> c)
```

Adds another collection.

Example

```
List<Integer> list1 = new ArrayList<>();

list1.add(10);
list1.add(20);

List<Integer> list2 = new ArrayList<>();

list2.add(30);
list2.add(40);

list1.addAll(list2);
```

```
System.out.println(list1);
```

Output

```
[10,20,30,40]
```

Time Complexity

Depends upon implementation.

Generally

```
O(n)
```

---

3. remove()

Signature

```
boolean remove(Object o)
```

Removes first matching object.

Example

```
List<String> list = new ArrayList<>();

list.add("A");
list.add("B");
list.add("A");

list.remove("A");

System.out.println(list);
```

Output

```
[B, A]
```

Only first occurrence removed.

Interview Favorite ⭐⭐⭐⭐⭐

For removing by index

```
list.remove(1);
```

For removing by object

```
list.remove("ABC");
```

This confusion happens because of method overloading.

---

remove() Integer Confusion

Example

```
List<Integer> list = new ArrayList<>();

list.add(10);
list.add(20);
list.add(30);

list.remove(1);
```

Output

```
[10,30]
```

It removes **index 1**, not value 1.

To remove value

```
list.remove(Integer.valueOf(20));
```

---

4. removeAll()

Signature

```
boolean removeAll(Collection<?> c)
```

Removes all matching elements.

Example

```
List<Integer> list =
Arrays.asList(1,2,3,4,5);

List<Integer> remove =
Arrays.asList(2,4);

list.removeAll(remove);
```

Output

```
[1,3,5]
```

---

5. retainAll()

Interview Favorite ⭐⭐⭐⭐

Keeps only common elements.

Signature

```
boolean retainAll(Collection<?> c)
```

Example

```
List<Integer> list1 =
new ArrayList<>(Arrays.asList(1,2,3,4));

List<Integer> list2 =
Arrays.asList(2,4,6);

list1.retainAll(list2);

System.out.println(list1);
```

Output

```
[2,4]
```

Think of

```
Intersection
```

---

removeAll() vs retainAll()

```
removeAll()

Difference

A - B
```

```
retainAll()

Intersection

A ∩ B
```

Interview Question ⭐⭐⭐⭐

---

6. contains()

Signature

```
boolean contains(Object o)
```

Example

```
List<String> list =
Arrays.asList("A","B","C");

System.out.println(list.contains("B"));
```

Output

```
true
```

Interview

How does contains() work?

Answer

Internally uses

```
equals()
```

Not

```
==
```

Hence overriding equals() is important.

---

7. containsAll()

Checks whether all elements exist.

Example

```
List<Integer> l1 =
Arrays.asList(1,2,3,4);

List<Integer> l2 =
Arrays.asList(2,3);

System.out.println(
l1.containsAll(l2)
);
```

Output

```
true
```

---

8. clear()

Removes everything.

```
list.clear();
```

Output

```
[]
```

Size becomes

```
0
```

---

9. isEmpty()

Checks whether collection is empty.

```
if(list.isEmpty())
```

Returns

```
true
```

or

```
false
```

Interview

Difference

```
list.size()==0
```

vs

```
list.isEmpty()
```

Answer

Both work.

Some implementations optimize

```
isEmpty()
```

Prefer

```
isEmpty()
```

---

10. size()

Returns number of elements.

Example

```
list.size();
```

Output

```
5
```

Interview

Maximum size?

Answer

```
Integer.MAX_VALUE
```

theoretically.

Implementation dependent.

---

11. iterator()

Inherited from Iterable.

Returns

```
Iterator<E>
```

Example

```
Iterator<String> itr =
list.iterator();

while(itr.hasNext())
{
    System.out.println(itr.next());
}
```

---

12. toArray()

Converts Collection to Array.

Example

```
Object[] arr =
list.toArray();
```

Java 11+

Preferred

```
String[] arr =
list.toArray(new String[0]);
```

Java 11+ optimization

```
String[] arr =
list.toArray(String[]::new);
```

---

13. stream()

Java 8

Converts Collection into Stream.

Example

```
list.stream()
    .filter(x->x>10)
    .forEach(System.out::println);
```

---

14. parallelStream()

Creates parallel stream.

Example

```
list.parallelStream()
```

Useful for CPU-intensive operations on large datasets.

---

15. removeIf()

Java 8 Feature

Signature

```
boolean removeIf(Predicate<? super E> filter)
```

Example

```
list.removeIf(
x -> x % 2 == 0
);
```

Removes all even numbers.

Output

```
[1,3,5]
```

---

16. forEach()

Java 8

Example

```
list.forEach(System.out::println);
```

Internally uses Consumer.

---

17. spliterator()

Java 8

Returns

```
Spliterator<E>
```

Used by Stream API.

Supports

- Parallel Processing
- Efficient Traversal
- Divide and Conquer

We'll study Spliterator separately.

---

equals() in Collections

Interview Favorite ⭐⭐⭐⭐⭐

Example

```
List<String> l1 =
Arrays.asList("A","B");

List<String> l2 =
Arrays.asList("A","B");

System.out.println(
l1.equals(l2)
);
```

Output

```
true
```

Collections compare

- Size

- Order (for Lists)
- Elements using `equals()`

---

hashCode()

Every Collection has hashCode().

Contract

```
Equal Collections

↓

Equal HashCodes
```

Mandatory for HashSet and HashMap.

UnsupportedOperationException

Interview Favorite ⭐⭐⭐⭐

Example

```
List<Integer> list =
Arrays.asList(1,2,3);

list.add(4);
```

Runtime

```
UnsupportedOperationException
```

Why?

Because

```
Arrays.asList()
```

returns a **fixed-size list** backed by the original array.

Adding or removing elements changes the size, so it is not allowed.

To create a modifiable list:

```
List<Integer> list =
new ArrayList<>(Arrays.asList(1,2,3));

list.add(4);
```

Null Handling

Depends upon implementation.

Example

```
ArrayList
```

Allows multiple nulls.

```
HashSet
```

Allows one null.

```
TreeSet
```

Generally does not allow null because comparisons are required.

Time Complexity (General)

Method	Typical Complexity
add()	O(1) average
remove()	O(n)
contains()	O(n)

Method	Typical Complexity
size()	O(1)
clear()	O(n)
iterator()	O(1)
addAll()	O(n)
removeAll()	O(n × m) (implementation-dependent)
retainAll()	O(n × m) (implementation-dependent)
isEmpty()	O(1)

Note: These are general complexities. The exact complexity depends on the underlying implementation (`ArrayList`, `HashSet`, `LinkedList`, etc.).

Frequently Asked Interview Questions

Q1. What is the root interface of the Java Collections Framework?

`Collection`.

Q2. Does Map extend Collection?

No. `Map` is a separate hierarchy because it stores key-value pairs instead of individual elements.

Q3. Which method checks object equality?

`equals()`.

Methods like `contains()`, `remove(Object)`, and `containsAll()` rely on `equals()`.

Q4. Why does add() return boolean?

Because some implementations (like `HashSet`) may reject duplicate elements, in which case `add()` returns `false`.

Q5. Difference between remove() and removeAll()?

- `remove(Object)` removes the first matching element.
- `removeAll(Collection)` removes all elements that are present in the specified collection.

Q6. Difference between removeAll() and retainAll()?

- `removeAll()` performs a **set difference** (`A - B`).
- `retainAll()` performs a **set intersection** (`A ∩ B`).

Q7. Why is Arrays.asList() not fully modifiable?

Because it returns a **fixed-size list** backed by the original array. You can replace elements using `set()`, but you cannot change the list's size with `add()` or `remove()`.

Key Takeaways

- `Collection` is the root interface for `List`, `Set`, and `Queue`.
- It extends `Iterable`, enabling iteration and enhanced for-loops.
- Learn every method of the `Collection` interface thoroughly—interviewers frequently ask about their behavior.
- `contains()` and `remove(Object)` rely on `equals()`, not `==`.
- `retainAll()` performs intersection, while `removeAll()` performs difference.
- `Arrays.asList()` returns a fixed-size list, not a fully mutable `ArrayList`.
- Time complexity depends on the concrete implementation.

Next Chapter

🚀 Chapter 3 – List Interface (Complete Deep Dive)

We'll cover:

- What is List?
- Characteristics of List
- Ordered vs Unordered Collections
- Duplicate Elements
- Positional Access
- List-specific Methods
- `ArrayList` vs `LinkedList`
- `ListIterator`

- Internal Working
- Time Complexity
- Real-world Use Cases
- Interview Questions